

HIERARCHICAL FAILURE CONTAINMENT FOR ROBOT SYSTEMS

Anonymous authors

Paper under double-blind review

ABSTRACT

Robot stacks are often organized hierarchically, but their failure handling is not. A local fault can leak upward through perception, planning, and execution layers until the whole mission is degraded or the system enters a brittle global fallback. We argue that the central design problem is not only detecting or repairing faults, but containing their propagation across hierarchy levels. We propose CONTAINNET, a containment-budget view of robot hierarchies in which each level is allowed only a bounded amount of local retry and a bounded summary of failure state before escalation. The goal is to minimize failure blast radius, not merely maximize eventual success. A simple simulator shows that explicit containment budgets reduce average propagation depth relative to unconstrained escalation, while preserving high success in the tested regime. We position the idea against robot fault diagnosis, fault-tolerant control, runtime assurance, behavior-tree recovery, and classical fault containment theory, and we identify the robotics-specific assumptions that those lines of work usually leave implicit.

1 INTRODUCTION

Hierarchical robot systems are attractive because they turn complex embodied tasks into manageable parts: low-level controllers stabilize motion, mid-level skills solve reusable subtasks, and high-level supervisors coordinate long-horizon behavior. Unfortunately, the hierarchy that helps organize behavior can also become a channel through which failure propagates. A grasp slip can trigger repeated retries, retries can poison task state, task state can trigger wrong replanning, and the supervisory layer can end up making brittle global decisions.

Most prior work responds to this problem with one of four moves: diagnose the fault, compensate the fault, filter unsafe actions, or add recovery branches. Those moves matter, but they treat recovery as the only design goal. We argue that *containment* is the missing central mechanism: the system should be designed so that failures stay local unless escalation is genuinely needed.

This paper makes three claims.

- Failure propagation across robot hierarchy levels is a real and measurable design variable.
- Explicit containment budgets, rather than unlimited local retry or blind upward bubbling, are a useful way to control that variable.
- The right novelty target is robotics-specific containment policy, not another generic monitor or planner.

The contribution here is intentionally architectural. We do not claim a universal theorem about every robot hierarchy. We instead propose a simple mechanism, test it in a toy simulator, and use the literature sweep to define the novelty boundary honestly.

2 RELATED WORK

2.1 CLASSICAL FAULT CONTAINMENT

Distributed systems and self-stabilization established the language of fault containment: locality, containment time, and the idea that a fault should not corrupt the whole system. Classic work on

fault containment, probabilistic containment, local stabilization, and containment-preserving protocol transformations shows that bounded failure spread can be formalized and sometimes proved.

The robotics gap is that a robot stack is not only a distributed protocol. It is hybrid, physical, partially observed, and often organized into a task hierarchy with distinct authority levels. A robot fault can therefore corrupt not just state but also the control flow that interprets state.

2.2 FAULT DIAGNOSIS AND FAULT-TOLERANT CONTROL

Robotics has deep literatures on fault diagnosis, sensor-fault accommodation, fault hiding, and active fault-tolerant control. These methods typically assume that faults can be estimated or compensated once they are detected. That assumption is useful, but it treats the robot body or controller as the main containment boundary. It does not tell us how to prevent an execution-layer fault from leaking into task-level reasoning.

2.3 BEHAVIOR TREES AND RUNTIME ASSURANCE

Behavior trees, supervisory monitors, and runtime assurance frameworks already provide hierarchical execution structure and online intervention. They are the closest implementation substrate for this paper. However, most of them still optimize for eventual success or safety, not for explicit failure containment across levels. A recovery branch may work well locally while still creating excessive upward propagation.

2.4 MULTI-ROBOT CONTAINMENT CONTROL

Containment control in multi-robot systems is a different but related meaning of containment: agents converge to a set or formation defined by leaders or boundaries. That literature shows that containment is a powerful control objective. Our thesis is complementary: in robot hierarchies, containment should also mean preventing failure from climbing the authority stack.

2.5 WHY CURRENT WORK IS STILL NOT ENOUGH

The papers above often assume that one of the following is true:

1. local recovery is always sufficient;
2. the supervisory layer can safely see all low-level failures;
3. failure escalation is cheap;
4. hierarchy is merely organizational, not protective.

Our paper breaks those assumptions by making escalation a budgeted, architecture-level decision.

3 A CONTAINMENT-BUDGET VIEW OF ROBOT HIERARCHIES

Consider a robot stack with levels $L1$ through LH , where $L1$ is the closest to physical actuation and LH is the mission supervisor. Each level can attempt local recovery, but each level has a bounded containment budget b_h . Once a level exhausts its budget, it emits a summarized failure token upward rather than a raw, potentially corrupt state dump.

The central idea is that the robot hierarchy should implement three rules:

- **Locality:** faults should be handled at the lowest viable level.
- **Quarantine:** higher levels should not inherit unbounded low-level failure detail.
- **Budgeted escalation:** upward propagation is allowed only after a bounded amount of local effort.

This mechanism is different from a verifier or a planner because the key object is not correctness proof at a single boundary. It is the control of *blast radius* across the stack.

Retries	Success rate	Avg. propagation	Avg. contained
0	0.998	0.4620	0.0000
1	1.000	0.1750	0.3895
2	1.000	0.0595	0.5330
3	1.000	0.0215	0.5855

Table 1: Toy simulator results. Containment budgets reduce upward propagation in this model.

3.1 A SIMPLE METRIC

Let P be the propagation depth of a fault event, defined as the number of hierarchy levels that receive nontrivial failure state before the event is resolved or quarantined. Let S be mission success. A containment policy is desirable if it reduces $\mathbb{E}[P]$ without collapsing S .

4 TOY SIMULATION

We implemented a small stochastic simulator in `scripts/sim_containment.py`. The simulator models a four-level hierarchy where each level may experience a fault, attempt local retries, and either contain the failure or escalate it upward.

Table 1 reports the aggregate behavior across 2,000 episodes for several local retry budgets.

The directional result is simple: modest local retry already cuts propagation sharply. The important point is not that retries are always good. The point is that containment is measurable and controllable, and thus can be designed rather than hoped for.

5 DISCUSSION

5.1 WHAT IS ACTUALLY NOVEL HERE

The novelty is the change in center of gravity. Existing work asks how to diagnose, compensate, or safely recover. This paper asks how to *bound the spread of failure* through a robot hierarchy. That shift matters because it changes what counts as success.

5.2 WHAT THE PAPER DOES NOT SOLVE

We do not prove optimality of a universal containment policy. We do not show that every robot stack should use the same budgets. We do not replace runtime monitors, behavior trees, or fault-tolerant control. Instead, we provide a vocabulary and a mechanism for composing those tools so that failure propagation is designed, not accidental.

5.3 WHEN THE IDEA IS WEAK

The idea is weaker if the robot stack is flat, if the hierarchy is purely decorative, or if the environment is so benign that failure escalation never matters. It is also weaker if the evaluation never leaves a toy simulator. Those are real limitations.

6 CONCLUSION

Hierarchical robot systems need more than recovery mechanisms. They need failure containment. We proposed a containment-budget view that treats upward propagation as a first-class cost, showed a simple simulation where budgets reduce propagation, and mapped the idea against the strongest hostile prior work. The result is not a finished universal theory, but a concrete robotics direction whose central mechanism is different from the usual suspects.

ACKNOWLEDGMENTS

Omitted for anonymity.

REFERENCES

- Ahmad et al. A behavior trees and motion generators approach to failure management, 2024. Referenced as a behavior-tree recovery baseline.
- Unknown. Fault tolerance and failure containment, 2005. Prior work referenced in the literature sweep.
- Unknown. Probabilistic fault-containment, 2007a. Prior work referenced in the literature sweep.
- Unknown. Robust fault detection and isolation in mobile robot, 2007b. Fault diagnosis baseline.
- Unknown. Fault-tolerant containment control for multi-uav systems based on adp, 2021. Containment control baseline.
- Unknown. Application of distributed containment control to multi-robot systems, 2022. Containment control baseline.
- Unknown. Runtime verification and field testing for ros-based robotic systems, 2024. arXiv preprint referenced in the literature sweep.
- Unknown. Aeros: A single-agent operating architecture with embodied capability modules, 2026a. ArXiv preprint referenced in the literature sweep.
- Unknown. Federated single-agent robotics: Multi-robot coordination without multi-agent fragmentation, 2026b. ArXiv preprint referenced in the literature sweep.
- Wu et al. Automated behavior tree error recovery framework for robotic systems, 2021. Referenced as a behavior-tree recovery baseline.